

Department of Computer Engineering and Information

Faculty of Science and Technology, Middlesex University Dubai



CST1510
**Programming for Data Communication
and Networks**

Assessment 2 Report

Isha Patel
MISIS: M01087113

Module Lecturer: Ms. Sumitra Kotipalli

GitHub Repository: https://github.com/Isha151/CW2_CST1510_M01087113_Week_7_12

Table of Contents

1. Introduction
2. How I Built the System
 - 2.1. Security and Database
 - 2.2. System Structure
 - 2.3. Code Organization (OOP Refactoring)
 - 2.4. Key Features
3. Reflection and Conclusion
 - 3.1. What I Learned
 - 3.2. Challenges I Faced
 - 3.3. Conclusion
 - 3.4. AI Usage Declaration
4. References

1. Introduction

This Multi Domain Intelligence Platform is an application developed using Streamlit that brings together multiple domains into a single system. The goal of the platform is to allow users to update, manage and analyse information from different domains through dashboards and simple analytics. This platform combines structured data management with visual insights and basic AI assistance.

The system allows the users to log in and access different dashboards depending on their required task. These dashboards include metadata management, IT support ticket management, and cyber incidents management. Each dashboard also has features for CRUD function (Add, update, and delete queries), visualization for better analytics, and lastly a simple AI assistant. The AI assistant has been given a prompt which matches each of the 3 domains, so it can deliver more accurate results relative to the requirements of the user. This project implements a Tier 2 system, as it includes analytics, authentication, and AI based assistance. The use of Object Oriented Programming (OOP) makes the system more structured and scalable when compared to a basic Tier 1 application.

2. How I Built the System

2.1 Security and Database: As with most applications, user security was an important factor when building this system. User passwords should never be stored in plain text, which is why when a user registers, their password must be converted into a hashed value (Ibm.com, 2025).

```
def register_user(username, password, role='user'):
    conn = connect_database()
    try:
        if get_user_by_username(conn, username):
            return False, f"Username '{username}' already exists."

        password_bytes = password.encode('utf-8')
        salt = bcrypt.gensalt()
        hashed = bcrypt.hashpw(password_bytes, salt).decode('utf-8')

        insert_user(conn, username, hashed, role)
        return True, f"User '{username}' registered successfully."
    except Exception as e:
        return False, f"Registration failed: {e}"
    finally:
        conn.close()
```

This hashing function works by first encoding the user's password into bytes, as bcrypt requires the password to be in byte format. Then a random salt key is generated using the `bcrypt.gensalt()` function. The password bytes and the salt key are hashed together using `bcrypt.hashpw()`, creating a secure hashed value. Lastly, the hashed password is decoded back into a string to store safely in a database. Hashing is used to protect user passwords by ensuring they are never stored in plain text. It also ensures that even if someone gained access to the database, they won't be able to read or reuse the original passwords.

The system uses an SQLite database to store all application data in a structured way (Ibm.com, 2023). Separate tables are used for Users, Datasets, IT Tickets, and Cyber Security Incidents. This makes sure that each type of data is stored independently and clearly. Separating these tables improves data organisation and makes updating records more efficient. The tables were created using predefined functions and their columns are designed to match the structure of the data being stored.

```
def create_users_table(conn):
    """Create the users table."""
    cursor = conn.cursor()
    sql = """
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT NOT NULL UNIQUE,
        password_hash TEXT NOT NULL,
        role TEXT DEFAULT 'user',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
    """
    cursor.execute(sql)
    conn.commit()
    print("✅ Users table created successfully!")
```

The function shown in the above code snippet is responsible for creating the Users table in the SQLite database. It follows a similar structure of every SQL query by starting with creating a database cursor. The SQL statement uses CREATE TABLE IF NOT EXISTS to ensure that the users table is only created if it does not already exist, preventing duplicate table creation. The table includes columns for a unique user ID, username. Hashed password, user role, and the account creation timestamp. Constraints such as NOT NULL and UNIQUE are applied to make sure usernames cannot be duplicated. After executing the SQL command, the changes are committed to the database. This function forms the foundation for access control features within the system.

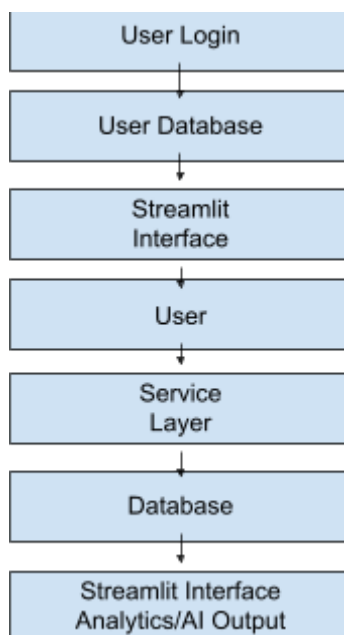
Below are the code snippets for the rest of the table creation functions. They follow a similar format as the create_user_table, differing in SQL statements and column creation.

```
def create_cyber_incidents_table(conn):
    """Create the cyber_incidents table."""
    cursor = conn.cursor()
    sql = """
    CREATE TABLE IF NOT EXISTS cyber_incidents (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        date TEXT,
        incident_type TEXT,
        severity TEXT,
        status TEXT,
        description TEXT,
        reported_by TEXT,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (reported_by) REFERENCES users(username)
    )
    """
    cursor.execute(sql)
    conn.commit()
    print("✅ Cyber Incidents table created!")
```

```
def create_datasets_metadata_table(conn):
    """Create the datasets_metadata table."""
    cursor = conn.cursor()
    sql = """
    CREATE TABLE IF NOT EXISTS datasets_metadata (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        dataset_name TEXT NOT NULL,
        category TEXT,
        source TEXT,
        last_updated TEXT,
        record_count INTEGER,
        file_size_mb REAL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
    """
    cursor.execute(sql)
    conn.commit()
    print("✅ Datasets Metadata table created!")
```

```
def create_it_tickets_table(conn):
    """Create the it_tickets table."""
    cursor = conn.cursor()
    sql = """
    CREATE TABLE IF NOT EXISTS it_tickets (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        ticket_id TEXT UNIQUE NOT NULL,
        priority TEXT,
        status TEXT,
        category TEXT,
        subject TEXT NOT NULL,
        description TEXT,
        created_date TEXT,
        resolved_date TEXT,
        assigned_to TEXT,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
    """
    cursor.execute(sql)
    conn.commit()
    print("✅ IT Tickets table created!")
```

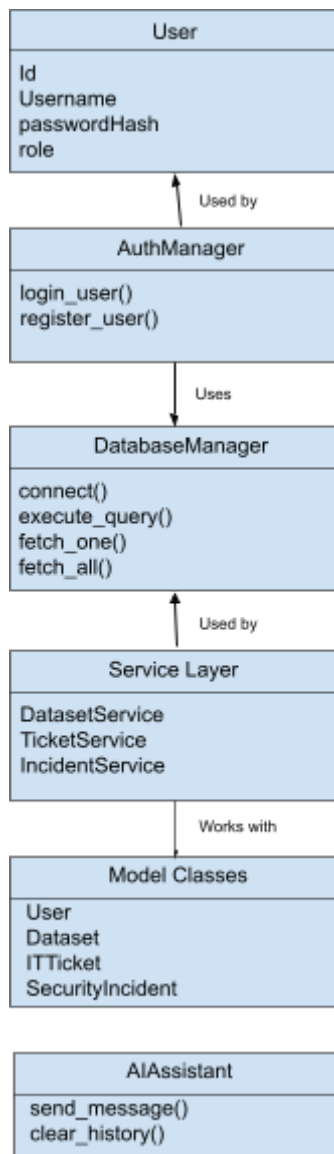
2.2 System Structure: The application follows a clear flow from user interaction to data storage and display. At a high level, the system works as such:



To begin with, the user either registers or logs in to the application. If the user registers, their information is stored securely inside the database. If the user logs in, their information is verified by the user info stored inside the database. Once authentication is completed, the user can access the Streamlit Interface. Through this interface, the user can perform actions such as creating, updating, or deleting records using the CRUD functionality, as well as requesting AI assistance. These requests are sent

from the Streamlit interface to the service layer which handles the application logic. The service layer then interacts with the database to perform the request. After the user's requests have been handled by the database, the results are sent back to the Streamlit Interface. The processed information is displayed to the user as tables, analytic dashboards, or AI generated assistance.

2.3 Code Organization (OOP Refactoring)



The system follows an Object Oriented design where responsibilities are clearly separated. The model classes (User, Dataset, ITTicket, and SecurityIncident) represent objects which store structured data used throughout the application (GeeksforGeeks, 2020).

The service layer has classes such as DatasetService, TicketService, and IncidentService. These services store the logic of the application and act as an interface between the Streamlit pages and the database. They use model objects to pass data in a clean and structured way.

All database operations are handled by the DatabaseManager class. This class establishes database connections and executes SQL queries.

User authentication is handled by the AuthManager, which works with the User model and the DatabaseManager to register users and verify their credentials securely.

The last essential class is an independent class `AIAssistant`. This class is used by the Streamlit pages to provide AI based assistance. It does not interact directly with the database, instead manages conversation history using session states.

This way of organization makes the project much cleaner and easier to manage. Since each class has a clear purpose, it reduces duplication of code and makes the logic easier to follow. This approach also helps increase readability as every page only has specific functions and remains relatively short.

Refactoring the code into classes improved the overall structure of the application. Now, if any new features are to be added they can easily be done so by creating new model and service classes. New features or domains can be added easily without changing the whole system. This makes the system more flexible and easier to extend.

2.4 Key Features: Charts and Visualizations:

Cyber Security:

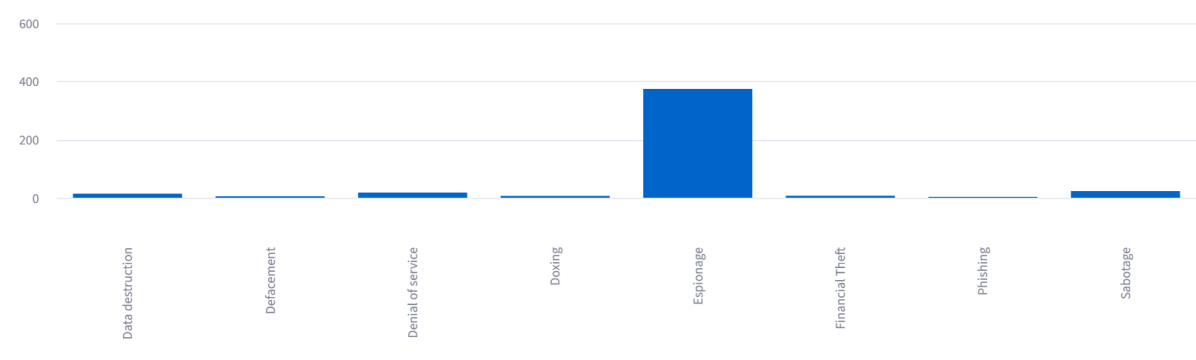
Cyber Incident Analytics

Total Incidents	High Severity	Medium Severity	Low Severity
484	0	481	3

This metrics shows the total number of incidents, incidents which are high, medium, and low Severity.

This helps the user easily summarise the dataset without having to go through each incident.

Incidents by Type



This graph is intended to show which Incident type has the greatest amount of incidents. This can be useful as the user can understand which incident type is happening more often than the others and act accordingly.

Incident Records

	severity	status	incident_type
0	Medium	Open	Espionage
1	Medium	Open	Espionage
2	Medium	Open	Data destruction
3	Medium	Open	Espionage
4	Medium	Open	Espionage
5	Medium	Open	Espionage
6	Medium	Open	Financial Theft
7	Medium	Open	Espionage
8	Medium	Open	Espionage
9	Medium	Open	Espionage

This is a table showing all of the incidents, but only with the columns of severity, status, and incident_type. This makes it very easy for the user to review all of the incidents.

*Data Science:***Dataset Analytics**

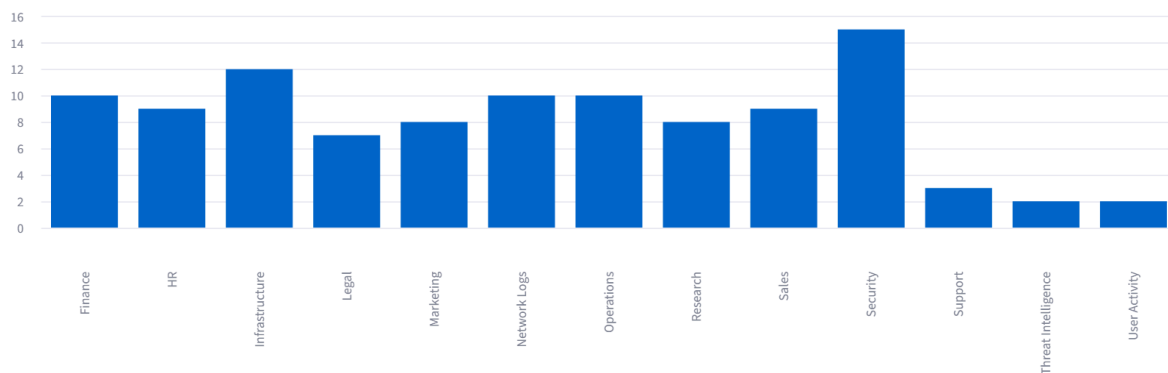
Total Datasets

105

Total Size (MB)

19005.4

This metric shows the total amount of datasets and size. This can be very useful for the user as it provides a quick overview of the dataset.

Datasets by Category

This graph is also really useful for the user as it displays which categories have the most number of datasets. The user does not need to check all the datasets and can find the information summarized in this chart.

Dataset Records

	id	dataset_name	category	source	last_updated	record_count	file_size_mb
0	1	Network Logs 2024	Network Logs	Firewall	2024-11-20	15000	45.5
1	2	Threat Intel Feed	Threat Intelligence	External Feed	2024-11-19	200	1.2
2	3	User Login History	User Activity	Active Directory	2024-11-20	5000	15
3	4	Malware Signatures	Security	Research Team	2024-10-30	1000	5.5
4	5	Customer Transactions	Finance	Payment Gateway	2024-11-18	50000	120.5
5	6	Employee Records	HR	HR System	2024-09-15	450	2.3
6	7	Server Health Metrics	Infrastructure	Monitoring Tool	2024-11-21	25000	60
7	8	Phishing Email Samples	Security	Email Gateway	2024-11-10	150	8.5
8	9	Network Logs 2024	Network Logs	Firewall	2024-11-20	15000	45.5
9	10	Threat Intel Feed	Threat Intelligence	External Feed	2024-11-19	200	1.2

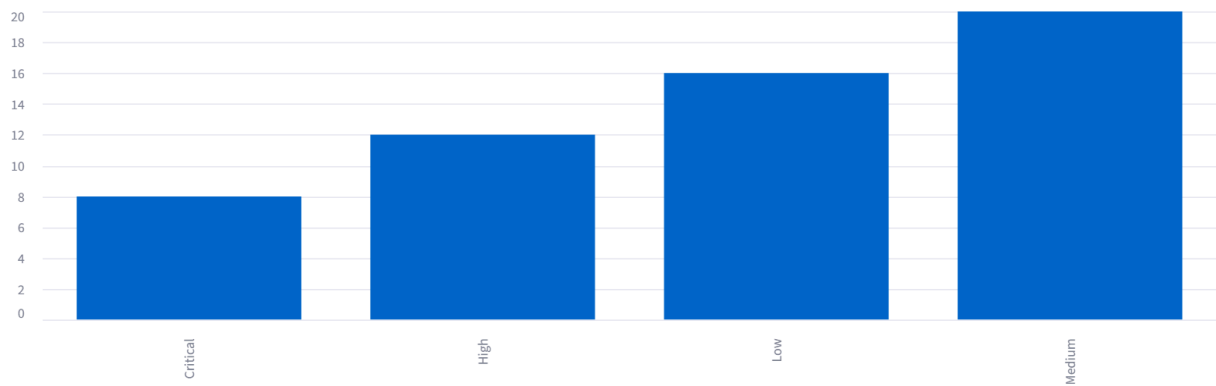
This table displays all the datasets and makes it easier for the user to look at all the information in one place.

IT Tickets:

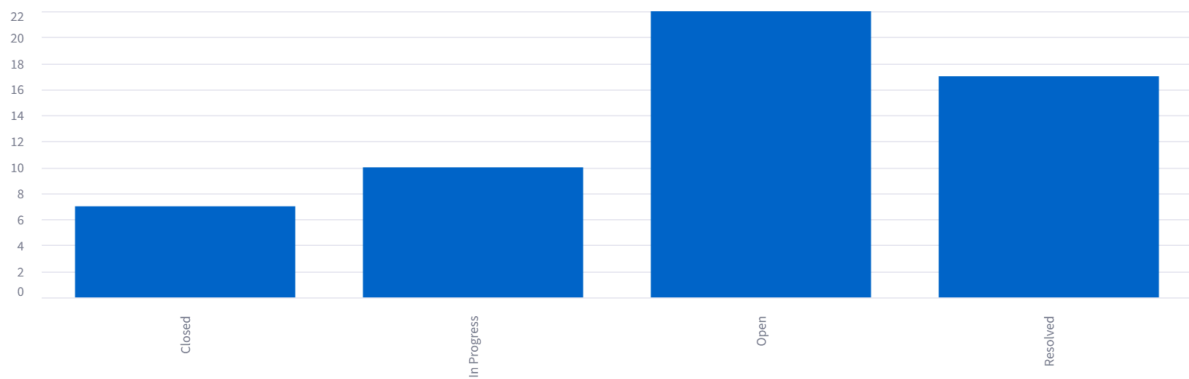
IT Ticket Analytics

Total Tickets	Open Tickets	Closed Tickets
56	22	7

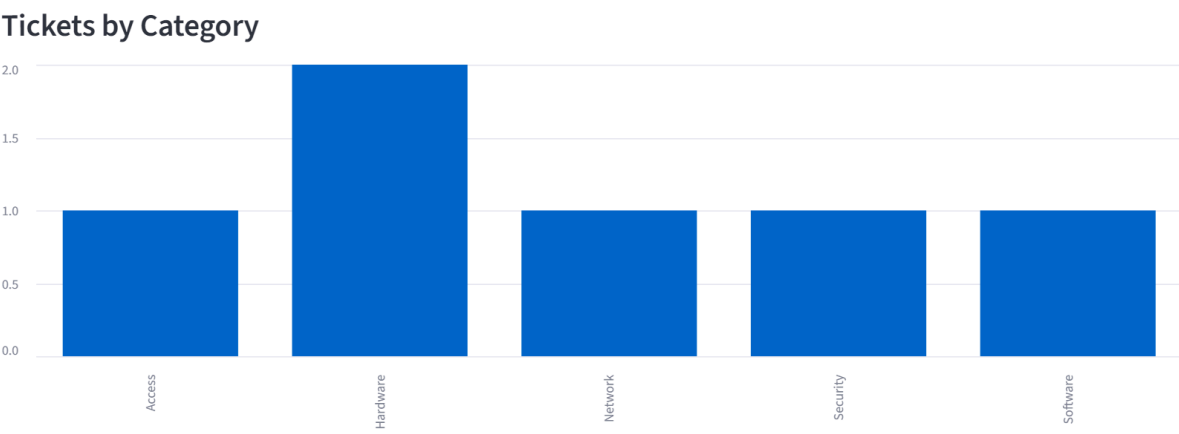
This metric provides a quick overview of the total number of tickets, as well as how many are currently open or closed. It helps the user instantly understand the overall workload and current system status without having to analyse each ticket.

Tickets by Priority

This bar chart shows how tickets are distributed based on different priority levels such as Critical, High, Medium, and Low. It helps the user quickly identify how many urgent issues are pending.

Tickets by Status

This chart shows the number of tickets in each status, including Open, In Progress, Resolved, and Closed. It makes it easier for the user to track progress.



This chart groups tickets based on categories such as Hardware, Software, Network, and Security. It helps users identify which types of issues happen the most frequent and where improvements might be needed.

Ticket Records

	priority	status	category
0	High	Open	Hardware
1	Medium	Resolved	Software
2	Low	In Progress	Network
3	Critical	Open	Security
4	Medium	Open	Access
5	Low	Closed	Hardware
6	High	Open	None
7	Medium	Resolved	None
8	Low	In Progress	None
9	Critical	Open	None

The ticket records table displays all ticket information in a structured way. It allows the user to review each ticket in detail.

AI Assistant:

Chat with AI Assistant

Ask about IT Tickets...



This AI Chatbot is included in every page. It helps the user ask questions about the page's specific domain. It is useful because if the user comes across a question they can easily look it up here. This makes the application easier to use and helps users quickly interpret analytics without manual investigation.

3. Reflection and Conclusion

3.1 What I learned: Through this project, the most valuable skill I gained was learning how to design and structure a full application using Object Oriented Programming. Instead of writing all the application logic in one file, I learned how to use a structured approach and create organized models, services, and user interface layers.

Another useful thing I learned was how to handle and manage data using the SQLite database. I also learned how to create interactive dashboards and visualizations that allow the users to quickly analyse data. Integrating the AI Chatbot using an external API helped me learn the features of other applications without having to code everything manually.

Overall, my understanding shifted from focusing only on making features work to organize the code in a way which is efficient and structured.

3.2 Challenges I Faced: One of the biggest challenges was refactoring the whole application to OOP.

Deciding which logic goes where and how to connect the models and services to the Streamlit Pages was a large challenge. This process helped me better understand separation of concerns and improved my ability to debug issues efficiently. Another challenge was managing application state. Particularly user login sessions and AI chat history. At first, the chat messages were shared across all pages. I solved this by using separate session state keys for each page, ensuring that each chatbot had its own conversation history.

3.3 Conclusion: In conclusion, this project created a multi-domain data platform with secure authentication, structured database design, interactive analytics dashboard, and AI chatbot integration. The structure is maintained by the use of the OOP system.

To improve the system further, I would add role based access to restrict certain features of the application based on user roles. I would also add time based analytics to track trends over time.

3.4 AI Usage Declaration: AI was used in this project to help debug and verify certain concepts. It was also used to clarify errors, understand why certain issues happened, and confirm whether specific approaches followed best practice. AI was not used to implement any requirements. All core functionality, design decisions, and implementation were completed by me. AI use was limited and only used for guidance and explanation.

4. References

GeeksforGeeks (2020). *Introduction of Object Oriented Programming*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/dsa/introduction-of-object-oriented-programming/>.

GeeksforGeeks (2022). *Hashing Passwords in Python with BCrypt*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/python/hashing-passwords-in-python-with-bcrypt/>.

Ibm.com. (2023). *IBM Robotic Process Automation*. [online] Available at: <https://www.ibm.com/docs/en/rpa/21.0.x?topic=connection-connect-sqlite> [Accessed 14 Dec. 2025].

Ibm.com. (2025). *IBM Netezza Performance Server*. [online] Available at: <https://www.ibm.com/docs/en/netezza?topic=set-hashing-functions>.